

Partie 2 — Les fichiers et les permissions

étendre sur l'application de la partie 1 la gestion des fichiers : en créer, les lister avec leurs droits, lire et modifier leur contenu quand on en a le droit, et partager des droits aux autres utilisateurs.

Terminé quand — deux utilisateurs différents peuvent se partager un fichier : le second voit le fichier du premier dans la liste, se voit refuser sa lecture, puis y accède dès que le premier lui accorde le droit `r`.

Les commandes à écrire

COMMANDE	EXIGE	EFFET
<code>ls -l</code>	aucun droit	Liste tous les fichiers : <code>rwd r--</code> propriétaire nomFichier
<code>touch <f></code>	aucun droit	Crée un fichier vide en <code>rwd ---</code> , dont le créateur est propriétaire
<code>cat <f></code>	droit r	Affiche le contenu du fichier
<code>nano <f></code>	droit w	Saisie multi-ligne qui remplace le contenu, jusqu'à une ligne valant EOF
<code>chmod <r w d> <f></code>	propriétaire	Donne un droit aux autres
<code>chmod -<r w d> <f></code>	propriétaire	Retire un droit aux autres

Livrer l'application en fichier .jar

- Ecrire — le fichier manifest.txt
- Construire le jar

Une session type, à deux utilisateurs

```
MINGW64/c/Users/Simplon/Documents/java-bootcamp/sprint1/briefs/LinePermission
Simplon@WIN-0DV0CLFCLV0 MINGW64 ~/Documents/java-bootcamp/sprint1/briefs/LinePermission
$ java -cp "target/classes;lib/jbcrypt-0.4.jar" ma.youcode.lineperm.Main
=====
LinPerm ? gestion de fichiers & droits
linperm> login
Login : abdelaziz
Mot de passe : 1234
Bienvenue abdelaziz !
abdelaziz@linperm> touche fichier1.txt
Commande inconnue. Tape 'help'.
abdelaziz@linperm> touch fichier1.txt
Fichier 'fichier1.txt' créé.
abdelaziz@linperm> nano fichier1.txt
--- Mode édition : fichier1.txt ---
(fichier vide)
--- Saisis ton texte. Tape EOF seul sur une ligne pour enregistrer. ---
Hello world
EOF
Fichier 'fichier1.txt' enregistré (1 ligne).
abdelaziz@linperm> ls -l
-rw-r--r-- mdidech test.txt
-rw-rw-r-- mdidech monFichier2.txt
-rw|--- sara monFichier1.txt
-rw|--- abdelaziz fichier1.txt
abdelaziz@linperm> cat test.txt
(fichier vide)
abdelaziz@linperm> cat monFichier1.txt
Permission denied.
abdelaziz@linperm> cat fichier1.txt
Hello world
abdelaziz@linperm> chmod r fichier1.txt
fichier1.txt : rw|--- -> rw-r--r--
abdelaziz@linperm> ls -l
-rw-r--r-- mdidech test.txt
-rw-rw-r-- mdidech monFichier2.txt
-rw|--- sara monFichier1.txt
-rw-r--r-- abdelaziz fichier1.txt
```

Les règles

Contrôle d'accès

- **Une seule catégorie s'applique.** Propriétaire → bloc de gauche. Sinon → bloc de droite. Jamais les deux, jamais de cumul.
- **Aucun administrateur.** Le premier compte créé n'a aucun privilège sur les fichiers des autres.
- **Droit manquant** → **Permission denied.** et l'action n'est pas exécutée.
- **ls -l ne demande aucun droit.** Lister n'est pas lire : voir les fichiers et leurs droits est permis à tout utilisateur connecté. Seul le contenu exige **r**.

Création et écriture

- Le créateur devient propriétaire, avec les permissions initiales **rw|---**.
- Un nom de fichier déjà pris est refusé. Un nom contenant un chemin aussi : sans ce contrôle, on écrirait hors du dossier de données.
- **nano** ne crée pas de fichier — c'est le rôle de **touch**. Sur un fichier inexistant, il affiche une erreur.

- Le droit **w** est vérifié **avant** d'entrer en mode édition, pour ne pas faire saisir un texte qui serait refusé à la fin.

Partage de droits

- Seul le **propriétaire** peut modifier les droits de son fichier.
- **chmod** n'agit que sur le bloc « autres ». Les droits du propriétaire ne bougent jamais.
- Le partage vise **tous les autres**, jamais une personne nommée : donner un droit à un seul utilisateur casserait le format **rwd|r--**.
- Redonner un droit déjà accordé n'est pas une erreur : rien ne change, et l'application le dit sans bloquer.

Persistence

- Les **droits** tiennent sur une ligne par fichier : **nom;proprietaire;rwd;r--**.
- Le **contenu** est multi-ligne : il vit dans de vrais fichiers, dans un dossier **data/**. Le mettre dans la ligne de droits obligerait à échapper les retours à la ligne.
- Tout est sauvegardé à la fin de chaque action réussie, et rechargé au lancement.

Le cas limite à traiter : ``rwd|-w-``, écrire sans pouvoir lire. Cet état est atteignable avec **chmod w**. Quand un utilisateur ouvre alors **nano**, le contenu actuel ne doit pas lui être montré, sinon **nano** devient un moyen de contourner le refus de **cat**. Il édite à l'aveugle.

Les fichiers à produire

FICHIER	RÔLE
model/FichierProtege.java	Un fichier : son nom, son propriétaire, et six booléens pour les droits
access/ControleAcces.java	La règle des catégories. Deux méthodes, aucun affichage.
service/FileService.java	Lister, créer, lire, écrire, changer les droits, sauvegarder
ui/ConsoleApp.java	à compléter — les six nouvelles commandes

Où placer le contrôle d'accès. Il appartient à **FileService**, pas à **ConsoleApp**. Si l'affichage décide des autorisations, il suffit d'oublier un **if** en ajoutant une commande pour ouvrir un trou. Placé dans le service, il est impossible de lire un fichier sans passer devant.

Critères d'acceptation

- ☐ Un fichier créé sort en **rwd|---** avec son créateur pour propriétaire.
- ☐ Le format **rwd|r-- propriétaire nomFichier** est respecté, trois positions par bloc.
- ☐ Un utilisateur voit dans **ls -l** les fichiers des autres, même sans aucun droit dessus.
- ☐ **cat** exige le droit **r** ; **nano** exige le droit **w**. Avoir l'un ne donne pas l'autre.
- ☐ Avec **w** mais sans **r**, **nano** masque le contenu actuel.

- ☐ Seul le propriétaire peut exécuter **chmod** sur son fichier.
- ☐ Repartager un droit déjà accordé n'affiche pas d'erreur bloquante.
- ☐ Le premier compte créé n'a aucun privilège sur les fichiers des autres.
- ☐ Toute action refusée affiche Permission denied et n'est pas exécutée.
- ☐ Un nom de fichier contenant un chemin est refusé.
- ☐ Droits et contenus survivent au redémarrage.

Compétences requises — partie 2

Tout ce qui a été acquis en partie 1 reste utilisé. Voici ce qui s'y ajoute.

Nouvelles notions

COMPÉTENCE	OÙ ELLE SERT
Méthodes `static`	ControleAcces.estAutorise(...) s'appelle sans new — la classe n'a aucun état
Surcharge de constructeur et appel this(...)	FichierProtege en a deux : le court délègue au long
Opérateur ternaire condition ? a : b	construire rwd à partir de trois booléens
Type `char` en paramètre	estAutorise(user, fichier, 'r')
StringBuilder et .append()	accumuler les lignes saisies dans nano
`List` / `ArrayList` , boucle for-each	parcourir les fichiers pour ls -l
.charAt(), .substring(), .startsWith()	lire un bloc de droits, détacher le tiret de chmod -r
Concaténation d'un char dans une chaîne	transformer un caractère en chaîne
Files.readString / writeString, Path.resolve()	lire et écrire le contenu des fichiers

Notions de conception

COMPÉTENCE	CE QUE ÇA SIGNIFIE
Séparation en couches	modèle → contrôle d'accès → service → interface. Les appels ne remontent jamais.
Une classe, une responsabilité	ControleAcces décide, FileService applique, ConsoleApp affiche
Convention de retour	null = accès refusé, chaîne vide = fichier vide. Deux choses différentes.
Méthodes `private` pour protéger	l'accès disque est privé, sinon on contournerait le contrôle